Infosys Interview Question Bank

Comprehensive Guide for .NET & React Developer Role

1. Object-Oriented Programming (OOPs) - Basics

- What are the four fundamental pillars of Object-Oriented Programming?
- How does Encapsulation differ from Abstraction in practical application?
- Can you explain Polymorphism with a real-world example?
- What is the difference between early binding and late binding?
- Explain the concept of Inheritance and the types of inheritance supported in C#.
- Why does C# not support multiple class inheritance, and how is it achieved?
- What is an Abstract class, and when should you use it over an Interface?
- How do class and object differ fundamentally in memory?
- What is a constructor, and what are the different types available in C#?
- Explain the purpose of a destructor and how it relates to garbage collection.
- What is an object in OOP, and how is it instantiated in C#?
- Explain the difference between a default constructor and a parameterized constructor.
- Can we overload a constructor in C#? Provide a practical example.
- What is a copy constructor, and does C# provide a default one?
- What are static constructors, and at what point in the lifecycle are they executed?
- How does the 'this' keyword work within a class instance?
- Can we force garbage collection in C# using GC.Collect()? Is it recommended?
- What is the difference between a shallow copy and a deep copy of an object?
- Explain the concept of message passing between objects in an application.
- What is the difference between coupling and cohesion in software design?

2. Object-Oriented Programming (OOPs) - Advanced

- What is method hiding, and how does the 'new' keyword function in this context?
- Explain the difference between method overloading and method overriding.
- What is a sealed class, and in what scenarios would you explicitly seal a class?
- How do access modifiers (public, private, protected, internal, protected internal) work in C#?
- What is the purpose of the 'virtual' keyword in a base class?
- Can an Interface contain method implementations? Explain default interface methods in modern C#.
- How do you prevent a specific method from being overridden in a derived class?
- Explain the "Is-A" vs "Has-A" relationship in class design.
- What is the diamond problem in multiple inheritance, and how does C# avoid it?
- How does composition compare to inheritance, and why is composition often preferred?

3. SOLID Design Principles

- What does the Single Responsibility Principle mean, and how does it prevent code smell?
- Explain the Open/Closed Principle. How do you write code that is open for extension but closed for modification?
- What is the Liskov Substitution Principle? Provide an example of its violation.
- How does the Interface Segregation Principle help in creating modular architecture?
- Explain the Dependency Inversion Principle. How does it relate to Dependency Injection?
- Why are SOLID principles critical in an enterprise application environment?
- Can you provide a scenario where strictly following SOLID might be overkill?
- How does the DRY (Don't Repeat Yourself) principle complement SOLID?
- What is the KISS (Keep It Simple, Stupid) principle in software design?
- How do design patterns relate to the implementation of SOLID principles?

4. C# Fundamentals

- What is the difference between Value types and Reference types in C#?
- Where are Value types and Reference types stored in memory?
- Explain the concepts of Boxing and Unboxing. Why should they be avoided for performance?
- What are the differences between 'ref' and 'out' parameters?
- How does the 'dynamic' type differ from the 'var' keyword?
- What is the difference between String and StringBuilder? When should you use which?
- Explain the difference between constant ('const') and read-only ('readonly') variables.
- What are nullable types, and how do you use the null-coalescing operator (??)?
- What is the difference between string interpolation and String.Format?
- Explain the difference between Convert.ToString() and .ToString().

5. C# Intermediate Concepts

- What is an Enum, and how do you specify underlying types for Enums?
- What are extension methods, and what are the strict rules for defining them?
- Explain the concept of Generics in C#. How do they improve performance and type safety?
- What are partial classes, and in what specific scenarios (like Entity Framework) are they useful?
- How does the 'yield' keyword work, and what is its relationship with IEnumerable?
- What is a Tuple, and how does C# 7+ support tuple deconstruction?
- Explain the difference between 'is' and 'as' operators.
- What is the purpose of the 'static' keyword for classes and constructors?
- What is a C# record, and how does it differ from a standard class?
- Explain the concept of indexers in C# and when to use them.

6. Delegates and Events (C#)

- What is a Delegate in C#? How is it analogous to a function pointer in C++?
- Explain the difference between singlecast and multicast delegates.
- What are the built-in Func, Action, and Predicate delegates? Provide usage examples.
- What is an Event? How does it differ from a standard Delegate?
- Why is it a best practice to check if an event is null before invoking it?
- What are Anonymous methods, and how did they evolve into Lambda expressions?
- Explain how event subscriptions can cause memory leaks in C# and how to prevent them.
- How do you pass a method as a parameter using delegates?
- What is the difference between a delegate and an interface?
- Explain the standard EventHandler and generic EventHandler<TEventArgs> delegates.

7. Exception Handling in C#

- Explain the try-catch-finally block structure. Will 'finally' always execute?
- What is the difference between 'throw' and 'throw ex' in a catch block?
- How do you create and use custom exceptions in C#?
- What are Exception filters (the 'when' keyword in catch blocks) introduced in C# 6?
- How does the Common Language Runtime (CLR) handle unhandled exceptions?
- Explain the AggregateException class and where it is typically encountered.
- What are the best practices for logging exceptions in an enterprise application?
- What is a StackOverflowException, and can it be caught in a standard try-catch block?
- How does the 'using' statement internally utilize try-finally blocks?
- Explain how inner exceptions are used and retrieved when wrapping exceptions.

8. LINQ (Language Integrated Query)

- What is LINQ, and why is it preferred over traditional foreach loops for data manipulation?
- What is the difference between Deferred Execution and Immediate Execution in LINQ?
- Explain the difference between IEnumerable and IQueryable interfaces.
- How do you perform a Left Outer Join using LINQ?
- What is the difference between Single(), SingleOrDefault(), First(), and FirstOrDefault()?
- Explain the Select and SelectMany projection operators.
- How do you use the GroupBy and OrderBy clauses in LINQ query syntax and method syntax?
- What are the performance implications of using complex LINQ queries on large collections?
- Explain the difference between the Any() and All() operators.
- What is the difference between Skip() and Take() and how are they used for pagination?

9. Asynchronous Programming in C#

- Explain the async and await keywords. How do they prevent thread blocking?
- What is a Task in C#, and how does it differ from a Thread?
- Explain the difference between Task.Run() and Task.Factory.StartNew().
- Why is returning 'void' from an async method considered a bad practice (except for event handlers)?
- What is a Deadlock in async programming, and how does ConfigureAwait(false) prevent it?
- How do you handle exceptions in async methods that return a Task versus void?
- Explain Task.WhenAll() vs Task.WaitAll().
- What are CancellationTokens, and how are they implemented in long-running tasks?
- Explain how async streams (IAsyncEnumerable) work in modern C#.
- What is ValueTask, and in what performance-critical scenarios should you use it over Task?

10. Multithreading and Concurrency

- What is the Thread Pool in .NET, and what are its advantages?
- Explain the concept of Race Conditions and how to mitigate them.
- What is the 'lock' statement in C#, and what is a Monitor?
- Explain the difference between Mutex and Semaphore. When would you use Mutex?
- What are concurrent collections in C# (e.g., ConcurrentDictionary), and why are they needed?
- What is the volatile keyword, and how does it affect compiler optimizations?
- Explain the Interlocked class and its role in atomic operations.
- How does the BackgroundWorker class differ from modern Task-based patterns?
- What is the ReaderWriterLockSlim class used for in high-concurrency environments?
- Explain the functional difference between Thread.Sleep() and Task.Delay().

11. Memory Management and Garbage Collection

- How does the .NET Garbage Collector (GC) determine which objects to clean up?
- Explain the generational model (Gen 0, Gen 1, Gen 2) of the Garbage Collector.
- What is the difference between Managed Code and Unmanaged Code?
- Explain the role of the IDisposable interface and the 'using' statement.
- What is the difference between Finalize (destructor) and Dispose?
- How does the GC.Collect() method work, and why is calling it manually usually discouraged?
- What is a Memory Leak in .NET, and what are common causes despite having a GC?
- Explain the concept of a Weak Reference in .NET.
- What is the LOH (Large Object Heap), and how does it differ from the SOH?
- Explain the purpose of GC.SuppressFinalize() inside the Dispose pattern.

12. .NET Framework & .NET Core Architecture

- What are the fundamental differences between .NET Framework and .NET Core?
- Explain the Common Language Runtime (CLR) and its key responsibilities.
- What is the Common Type System (CTS) and Common Language Specification (CLS)?
- Explain the execution process from source code to machine code (IL code, JIT compiler).
- What is .NET Standard, and how does it solve cross-platform compatibility issues?
- Explain the anatomy of an Assembly in .NET (Manifest, IL, Metadata, Resources).
- What is the Global Assembly Cache (GAC), and does it exist in .NET Core?
- How does cross-platform execution actually work in .NET Core?
- What is CoreCLR, and how does it compare to the traditional .NET Framework CLR?
- Explain the role of the dotnet CLI toolchain in modern .NET development.

13. ASP.NET Core Fundamentals

- Explain the request processing pipeline in ASP.NET Core.
- What is Middleware in ASP.NET Core, and how do you configure it in Program.cs?
- Explain the sequence and importance of ordering in the Middleware pipeline.
- What is Kestrel, and why is it used as the default web server in ASP.NET Core?
- Can Kestrel be used without a reverse proxy like IIS or Nginx? Explain the pros and cons.
- What is the significance of the appsettings.json file, and how does environment-based configuration work?
- Explain how Routing works in ASP.NET Core (Convention-based vs. Attribute routing).
- How do you configure and serve static files (HTML, CSS, JS) in ASP.NET Core?
- What is the difference between IApplicationBuilder and IEndpointRouteBuilder?
- Explain the concept of application Environments (Development, Staging, Production).
- What is the role of the IHostedService interface in creating background tasks?
- Explain the concept of Output Caching and how it is implemented in .NET Core.
- How do you manage Application Secrets safely during local development without committing them to source control?
- What is the Options Pattern, and how do you bind appsettings.json configurations to strongly typed C# classes?
- Explain the difference between In-Process and Out-Of-Process hosting models.
- How do you handle global exception handling using the UseExceptionHandler middleware?
- What are rate limiting and throttling, and how can they be configured in ASP.NET Core 7/8?
- Explain how Health Checks are implemented and monitored in a .NET Core microservice environment.
- What is the architectural difference between AddControllers, AddControllersWithViews, and AddRazorPages?
- How does ASP.NET Core handle sessions, and where is the session state stored in a distributed application?

14. Dependency Injection in .NET Core

- What is Dependency Injection (DI), and why is it built directly into ASP.NET Core?
- Explain the three service lifetimes: Transient, Scoped, and Singleton.
- If a Singleton service injects a Transient service, what happens? (Captive Dependency problem).
- How do you register and resolve services in modern .NET Core (Program.cs)?
- What is the difference between AddScoped() and AddTransient() in the context of an HTTP request?
- How can you implement constructor injection versus method injection?
- Can you resolve dependencies inside an attribute or action filter? If so, how?
- How do you register multiple concrete implementations of the same interface?
- What is the IServiceProvider, and how is it used manually to resolve dependencies?
- Explain the concept of Keyed Services introduced in .NET 8.

15. ASP.NET Core MVC & Filters

- Explain the Model-View-Controller (MVC) architectural pattern in the context of ASP.NET Core.
- What are Action Filters, and at what stage of the pipeline do they execute?
- List the types of filters available in ASP.NET Core (Authorization, Resource, Action, Exception, Result).
- How do you create a custom Exception Filter globally?
- Explain Model Binding and how data is mapped from HTTP requests to controller parameters.
- What is Data Annotations, and how does ModelState.IsValid work for validation?
- What are Tag Helpers, and how do they improve upon traditional HtmlHelpers?
- How do View Components differ from traditional Partial Views?
- What is anti-forgery token validation, and how does it prevent CSRF?
- Explain how routing constraints are applied within route templates.

16. Entity Framework Core (EF Core)

- What is an ORM, and how does Entity Framework Core implement this pattern?
- Explain the Database-First versus Code-First approaches in EF Core.
- What is the DbContext class, and what are its main responsibilities?
- How do Migrations work in EF Core? Explain the commands used to apply them.
- Explain the difference between Lazy Loading, Eager Loading, and Explicit Loading.
- How do you configure Eager Loading using the 'Include' and 'ThenInclude' methods?
- What is tracking in EF Core, and when should you use AsNoTracking() to improve performance?
- How does EF Core handle concurrency conflicts?
- How do you implement a many-to-many relationship using Fluent API in EF Core?
- How does EF Core support and execute raw SQL queries against the database?

17. Web API and REST Principles

- What are the core principles of a RESTful Web API?
- Explain the differences and standard use cases for HTTP GET, POST, PUT, PATCH, and DELETE.
- What is the difference between PUT and PATCH regarding idempotency and resource updates?
- Explain standard HTTP Status Codes (e.g., 200, 201, 204, 400, 401, 403, 404, 500).
- What is Content Negotiation in Web API, and how does it determine the response format (JSON/XML)?
- How do you handle API Versioning in ASP.NET Core (Query string, URL path, Header)?
- Explain HATEOAS (Hypermedia as the Engine of Application State).
- How do you return standardized error responses (ProblemDetails) in Web API?
- Explain the concept and role of content formatters in ASP.NET Core.
- How do you secure Web API endpoints using HTTPS and HSTS?

18. Web API Security & JWT

- How does Token-based authentication (JWT) work in a stateless Web API?
- What are the three parts of a JSON Web Token (Header, Payload, Signature)?
- How do you validate a JWT token on incoming requests in ASP.NET Core?
- What is CORS (Cross-Origin Resource Sharing), and how do you configure it in .NET Core?
- Explain the difference between Authentication and Authorization.
- How do you implement Role-based and Policy-based authorization in controllers?
- What is SQL Injection, and how does EF Core or parameterized queries protect against it?
- How do you mitigate Cross-Site Scripting (XSS) and CSRF attacks in API endpoints?
- How do you implement Refresh Tokens alongside JWT to handle token expiration?
- What is the Data Protection API (DPAPI) in .NET Core, and what is it used for?

19. Microservices & Advanced Architecture

- What are the key characteristics of a Microservices Architecture compared to a Monolith?
- How do distinct microservices share data without violating service boundaries?
- Explain synchronous vs asynchronous inter-service communication (REST vs Message Brokers).
- What is an API Gateway, and what role does it play in a microservices ecosystem?
- Explain the CQRS (Command Query Responsibility Segregation) pattern.
- What is the Mediator pattern, and how does the MediatR library simplify controller logic?
- How do you handle distributed transactions and eventual consistency (e.g., Saga pattern)?
- Explain the Strangler Fig pattern for migrating from a monolith to microservices.
- What is Service Discovery, and how do tools like Consul or Eureka facilitate it?
- How do you manage centralized logging and tracing across microservices?

20. React.js Fundamentals

- What is React, and how does it differ from a traditional framework like Angular?
- Explain the concept of the Virtual DOM. How does it improve rendering performance?
- What is JSX, and why is it used instead of plain JavaScript?
- What are React Components, and what is the difference between Functional and Class components?
- Explain the concept of Props. Are props mutable or immutable?
- What is State in React, and how does it differ from Props?
- Why must you use setState() or state updater functions instead of mutating state directly?
- What is the purpose of the 'key' attribute when rendering lists in React?
- What is the significance of the root DOM node in a standard React application?
- How does React handle synthetic events differently from native DOM events?
- Explain the concept of unidirectional (one-way) data flow in React.
- What is the 'children' prop, and how is it used in component composition?
- How do you conditionally render components based on state or props in JSX?
- What is the difference between a controlled component and an uncontrolled component in form handling?
- Explain the purpose of React.Fragment and its short syntax (<></>).
- How do you pass data from a child component back up to a parent component?
- What are defaultProps and propTypes? Are they still relevant when using TypeScript with React?
- How does React handle DOM rendering differently when using Server-Side Rendering (SSR) via Next.js?
- What is the StrictMode component, and why does it intentionally render components twice in development mode?
- What is the difference between a presentational (dumb) component and a container (smart) component?

21. React Hooks and Lifecycle

- What are React Hooks, and what rules must you follow when using them?
- Explain the useState hook and how it handles complex objects or arrays.
- What is the useEffect hook? How does the dependency array control its execution?
- How do you simulate componentDidMount, componentDidUpdate, and componentWillUnmount using useEffect?
- What is the difference between useMemo and useCallback hooks?
- When should you use the useRef hook (DOM access vs preserving mutable values without re-renders)?
- How do you create a custom Hook, and why would you do so?
- How does useReducer differ from useState, and when should you choose it?
- What is the useLayoutEffect hook, and how does it differ from useEffect chronologically?
- Explain how to use the useImperativeHandle hook alongside forwardRef.

22. React Advanced Concepts

- What is "Prop Drilling," and how can you avoid it?
- Explain the Context API (createContext, Provider, useContext) for global state.
- What are Higher-Order Components (HOCs), and how do they enable logic reuse?
- What are Render Props, and how do they compare to HOCs?
- Explain Error Boundaries in React. Why can they only be implemented using Class components?
- What are React Fragments, and why are they preferred over wrapper div elements?
- What is the purpose of React.StrictMode in development?
- How do you handle controlled vs uncontrolled form components in React?
- What are Portals in React, and in what scenarios (like Modals) are they necessary?
- How does React 18's automatic batching optimize performance?

23. React State Management (Redux)

- What is Redux, and what are its three core principles?
- Explain the roles of Actions, Reducers, and the Store in the Redux architecture.
- Why must Reducers be pure functions? What happens if they mutate state?
- What is a Redux Middleware, and what problem does Redux Thunk or Redux Saga solve?
- Explain the difference between Context API and Redux. When would you choose one over the other?
- What is Redux Toolkit (RTK), and how does it simplify traditional Redux boilerplate (createSlice)?
- How does the useSelector hook optimize re-renders compared to mapStateToProps?
- What is the purpose of the combineReducers function in large applications?
- How do you approach testing Redux reducers and asynchronous actions?
- Explain the role of the Provider component in the React-Redux ecosystem.

24. React Routing and Optimization

- How does React Router enable Single Page Application (SPA) navigation without page reloads?
- Explain the difference between BrowserRouter and HashRouter.
- How do you extract URL parameters and query strings using React Router hooks (useParams)?
- What is Code Splitting in React, and how do React.lazy and Suspense implement it?
- How do you optimize a React application to prevent unnecessary component re-renders (React.memo)?
- What are Web Vitals, and how do you measure performance in a React application?
- Explain the Virtual DOM reconciliation process (Diffing algorithm).
- How does the useNavigate hook work for programmatic navigation in React Router v6?
- Explain the concept and implementation of nested routes.
- How do you implement protected or private routes that require user authentication?

25. Full-Stack Integration (.NET + React)

- How do you securely configure CORS in an ASP.NET Core API to accept requests from a React frontend?
- Explain how you would handle JWT token storage in a React application (Local Storage vs HttpOnly Cookies).
- How do you handle API request interceptors in React (e.g., using Axios) to attach bearer tokens?
- What is the standard architecture for deploying a React app and a .NET API (same server vs decoupled)?
- How do you handle routing fallback in ASP.NET Core when serving a compiled React SPA?
- Describe how you would implement real-time communication between .NET and React using SignalR.
- How do you handle different environment variables across a React build and a .NET runtime?
- Explain how to implement multi-part file uploads from a React component to a .NET API.
- What is the best way to handle timezone differences and dates between a React frontend and .NET backend?
- How do you handle global error capturing in React and logging those errors to a .NET API?

26. Logical Coding - Arrays & Strings (Part 1)

- Write a function to find the maximum and minimum elements in an unsorted integer array.
- How do you reverse a string in C# without using built-in library functions?
- Write an algorithm to determine if a given string is a palindrome.
- Given an array of integers, how do you find all pairs that sum up to a specific target number?
- Write a program to remove duplicate elements from an array or List efficiently.
- How do you find the second largest element in an array in a single traversal?
- Write code to count the frequency of each character in a given string using a Dictionary.
- Write a function to check if two strings are isomorphic.
- How do you find the first repeating element in an array of integers?
- Write code to calculate the sum of all elements in an array dynamically.

27. Logical Coding - Arrays & Strings (Part 2)

- Write a function to determine if two strings are anagrams of each other.
- How do you find the missing number in an integer array containing 1 to N?
- Write a program to move all zeros to the end of an array while maintaining the order of other elements.
- Given a string, find the first non-repeating character and return its index.
- Write a function to find the longest common prefix among an array of strings.
- How do you merge two sorted arrays into a single sorted array without using extra space?
- Write code to compress a string (e.g., "AABBBCC" becomes "A2B3C2").
- Write a function to find the length of the longest substring without repeating characters.
- How do you rotate an array to the right by k steps in-place?
- Write code to verify if a string contains exclusively numeric digits without using Regex.

28. Logical Coding - Mathematics & Logic

- Write a function to check if a given number is prime. Optimize for large numbers.
- How do you generate the nth number in the Fibonacci sequence (both iteratively and recursively)?
- Write a program to calculate the factorial of a number using recursion. What is a StackOverflow?
- Write code to swap two numerical variables without using a temporary third variable.
- Given an integer, reverse its digits. Handle negative numbers correctly.
- Write a function to check if a given year is a leap year.
- Write an algorithm to find the greatest common divisor (GCD) of two numbers (Euclidean algorithm).
- Write a program to find the power of a number (x^y) without using Math.Pow.
- How do you check if a given number is an Armstrong number?
- Write a function to generate Pascal's Triangle up to n rows and print it to the console.

29. Logical Coding - Searching and Sorting

- Explain the Binary Search algorithm. Write the code to perform binary search on a sorted array.
- What is the time complexity of Binary Search compared to Linear Search?
- Write a program to implement Bubble Sort. How can you optimize it to stop early if sorted?
- Write an algorithm for Selection Sort and explain its time complexity.
- Write code for Insertion Sort. In what scenarios is Insertion Sort preferred?
- Explain how Quick Sort works (partitioning). What is its worst-case time complexity?
- Write a function to implement Merge Sort and explain its divide-and-conquer approach.
- Explain Counting Sort and under what constraints it is applicable.
- Write code to implement Linear Search and discuss its space-time complexity.
- How do you write a custom comparer to sort an array of strings alphabetically or by length?

30. Logical Coding - Linked Lists & Data Structures

- Write the class structure for a singly linked list in C#.
- Write an algorithm to reverse a singly linked list iteratively.
- How do you detect if a linked list has a cycle (loop)? Explain Floyd's Cycle-Finding Algorithm.
- Write a function to find the middle node of a linked list in a single pass.
- Explain the difference between a Stack and a Queue. Provide real-world use cases for both.
- How do you implement a Queue using two Stacks?
- Explain how a HashTable (or Dictionary in C#) handles internal memory collisions.
- Write a function to evaluate a standard postfix expression using a Stack structure.
- How do you check for balanced and nested parentheses in a mathematical expression?
- Explain how a Binary Search Tree (BST) works and its traversal methods.

31. System Design and Clean Code

- Given the requirement to build an e-commerce shopping cart, which design patterns would you apply?
- How do you ensure secure storage of user passwords in a database?
- Explain the Repository Pattern and the Unit of Work pattern in C#.
- What are the characteristics of REST API endpoints that follow clean coding guidelines?
- How would you approach designing a scalable logging and monitoring system for microservices?
- If a database query is running slow, what steps do you take to identify and fix the performance bottleneck?
- Explain the concept of caching (Redis, MemoryCache) and when cache invalidation is necessary.
- Explain the concept of a Content Delivery Network (CDN) and when you should use one.
- How do you design a robust URL shortener service (like bit.ly)?
- What is the difference between horizontal scaling (scaling out) and vertical scaling (scaling up)?

32. HR and Behavioral Questions

- Why do you want to join Infosys as a Back End / Full Stack Developer?
- Describe a time when you had a disagreement with a team member over a technical decision. How did you resolve it?
- Explain a challenging technical problem you faced in a recent project and how you solved it.
- How do you stay updated with the latest changes in the .NET ecosystem and React framework?
- Describe your experience working in Agile/Scrum environments.
- What is your approach to reviewing code during Pull Requests?
- Where do you see your technical career path leading in the next three to five years?
- Describe a situation where you had to quickly learn a new technology to complete an urgent project.
- How do you handle working under tight deadlines and changing requirements?
- What are your greatest professional strengths and what is an area you are actively improving?

33. Advanced Memory Management & Performance

- How do you identify and diagnose a memory leak in a production .NET application?
- What profiling tools do you use for .NET Core (e.g., dotMemory, dotnet-dump, VS Profiler)?
- Explain the concept of ArrayPool<T> and how it helps minimize garbage collection pauses.
- What are Span<T> and Memory<T>, and how do they reduce heap allocations for string and array manipulations?
- How does the 'ref struct' constraint work, and why is it essential for stack-only types like Span<T>?
- What is the difference between stackalloc and heap allocation, and when is it safe to use stackalloc?
- How does Large Object Heap (LOH) fragmentation occur, and how can you force LOH compaction in modern .NET?
- Explain the hidden memory allocation impact of capturing local variables in lambda expressions (closures).
- What is string interning (String.Intern), and under what specific scenarios can it save memory?
- Walk through the exact implementation of the IDisposable pattern for a class managing unmanaged resources.